

RSA

Aspekt	Beschreibung
Definition von RSA	RSA (Rivest-Shamir-Adleman) ist ein asymmetrisches Verschlüsselungsverfahren, das auf der Schwierigkeit der Faktorisierung großer Zahlen basiert. RSA nutzt ein Schlüsselpaar, bestehend aus einem öffentlichen Schlüssel (zum Verschlüsseln) und einem privaten Schlüssel (zum Entschlüsseln).
Schlüsselgröße	2048 Bit: Weit verbreitet und als sicher gilt. 3072 Bit: Bietet eine höhere Sicherheit. 4096 Bit: Noch sicherer, aber langsamer in der Verarbeitung. RSA nutzt eine Schlüsselgröße, die die Sicherheit des Verfahrens bestimmt, da größere Schlüssel schwieriger zu knacken sind.
Funktionsweise	1. Schlüsselerzeugung: Ein Paar von Schlüsseln wird erzeugt, basierend auf der Faktorisierung zweier großer Primzahlen. 2. Verschlüsselung: Der öffentliche Schlüssel wird verwendet, um Daten zu verschlüsseln. 3. Entschlüsselung: Der private Schlüssel wird verwendet, um verschlüsselte Daten zu entschlüsseln.
Schlüsselerzeugung	1. Zwei große, zufällig gewählte Primzahlen p und q werden multipliziert, um $n = p \times q$ zu berechnen. n wird als Teil des öffentlichen und privaten Schlüssels verwendet. 2. Der Wert von $\phi(n)$ (Eulersche Phi-Funktion) wird berechnet. 3. Ein öffentlicher Exponent e wird gewählt, der teilerfremd zu $\phi(n)$ ist. 4. Der private Exponent d wird berechnet, so dass $d \times e \equiv 1 \pmod{\phi(n)}$.
Verschlüsselung	Um eine Nachricht M zu verschlüsseln, wird der öffentliche Schlüssel (e, n) verwendet: $C = M^e \pmod{n}$, wobei C der verschlüsselte Text ist und M die zu verschlüsselnde Nachricht.
Entschlüsselung	Der private Schlüssel (d, n) wird verwendet, um den verschlüsselten Text C zu entschlüsseln: $M = C^d \pmod{n}$, wobei M der entschlüsselte Text ist.
Modulararithmetik	RSA basiert auf der Modulararithmetik, insbesondere auf der Berechnung von $x^e \pmod{n}$ und $x^d \pmod{n}$, um die Nachricht zu verschlüsseln bzw. zu entschlüsseln. Diese Berechnungen sind effizient, selbst bei sehr großen Zahlen.
Sicherheitsaspekte	Schlüsselgröße: Größere Schlüssel bieten mehr Sicherheit. Faktorisierung der Zahlen: Die Sicherheit von RSA basiert auf der Schwierigkeit, n zu faktorisieren. Wenn die verwendeten Zahlen zu klein sind, kann der private Schlüssel leicht berechnet werden. Private Schlüssel schützen: Der private Schlüssel muss sicher aufbewahrt werden.
Verwendung von RSA in Java	RSA kann in Java über die Java Cryptography Extension (JCE) und die Klassen <code>KeyPairGenerator</code> , <code>Cipher</code> , und <code>KeyFactory</code> verwendet werden.

Aspekt	Beschreibung
Java-Klassen	<code>KeyPairGenerator</code> : Wird verwendet, um ein Schlüsselpaar (öffentlicher und privater Schlüssel) zu erzeugen. <code>Cipher</code> : Wird verwendet, um Daten zu verschlüsseln und zu entschlüsseln. <code>KeyFactory</code> : Wird verwendet, um einen Schlüssel aus einem Byte-Array zu erzeugen. <code>PublicKey</code> und <code>PrivateKey</code> : Schnittstellen für den öffentlichen und privaten Schlüssel.
Wichtige Methoden	<code>KeyPairGenerator.getInstance("RSA")</code> : Initialisiert den Schlüsselpaar-Generator für RSA. <code>KeyPairGenerator.initialize(int keysize)</code> : Legt die Schlüssellänge für RSA fest. <code>Cipher.getInstance("RSA")</code> : Initialisiert die <code>Cipher</code> -Klasse für RSA. <code>Cipher.doFinal(byte[] input)</code> : Verschlüsselt oder entschlüsselt die Daten. <code>KeyFactory.getInstance("RSA")</code> : Wird verwendet, um ein Schlüsselpaar zu erzeugen.
Schlüsselmanagement in Java	RSA-Schlüssel können aus Dateien geladen und gespeichert werden, wobei Base64 zur Kodierung und Decodierung verwendet wird. Es wird empfohlen, den privaten Schlüssel sicher zu speichern (z.B. in einem KeyStore).
Padding in RSA	RSA kann mit verschiedenen Padding-Schemata verwendet werden, wie: PKCS#1 v1.5: Ein älteres Padding-Schema, das immer noch weit verbreitet ist. OAEP (Optimal Asymmetric Encryption Padding): Ein neueres und sichereres Padding-Schema, das in modernen Anwendungen bevorzugt wird.
Digitale Signaturen	RSA kann auch zur Signierung von Daten verwendet werden, indem eine Nachricht mit dem privaten Schlüssel signiert wird. Um die Signatur zu überprüfen, wird der öffentliche Schlüssel des Absenders verwendet.
Performance	RSA-Verschlüsselung ist aufgrund der großen Zahlberechnungen langsamer als symmetrische Algorithmen wie AES. Daher wird RSA häufig in hybriden Systemen eingesetzt, bei denen RSA nur für den Schlüsselaustausch verwendet wird und die Datenverschlüsselung mit einem schnelleren symmetrischen Algorithmus erfolgt.
Angriffe auf RSA	Faktorisierung: RSA ist unsicher, wenn die Primzahlen zu klein sind. Mit der Entwicklung effizienter Faktorisierungsalgorithmen (z.B. Pollard's Rho) wird RSA mit kurzen Schlüsseln zunehmend anfällig. Man-in-the-Middle-Angriffe: Wenn der Schlüsselaustausch nicht sicher ist, könnte ein Angreifer den öffentlichen Schlüssel ändern.
Java-Bibliotheken	RSA wird in der Java Cryptography Extension (JCE) unterstützt, und das Framework ermöglicht den Einsatz von RSA in verschiedenen Szenarien, z.B. bei der Schlüsselgenerierung, Verschlüsselung und Signierung von Daten.

Methoden und Klassen

Methode/Klasse	Beschreibung
<code>KeyPairGenerator.getInstance("RSA")</code>	Diese Methode erstellt eine Instanz des <code>KeyPairGenerator</code> für den RSA-Algorithmus. Mit dieser Instanz kann ein Schlüsselpaar (öffentlicher und privater Schlüssel) erzeugt werden.
<code>KeyPairGenerator.initialize(int keysize)</code>	Mit dieser Methode wird die Schlüssellänge für das Schlüsselpaar festgelegt (z.B. 2048 Bit oder 4096 Bit). Eine größere Schlüsselgröße bietet eine höhere Sicherheit, beeinträchtigt jedoch die Leistung.
<code>Cipher.getInstance("RSA")</code>	Diese Methode erstellt eine Instanz der <code>Cipher</code> -Klasse für den RSA-Algorithmus. Sie ermöglicht die Verschlüsselung und Entschlüsselung von Daten mit dem RSA-Schlüsselpaar.
<code>Cipher.doFinal(byte[] input)</code>	Diese Methode führt die Verschlüsselung oder Entschlüsselung der Eingabedaten aus und gibt das Ergebnis zurück. Bei der Verschlüsselung mit RSA wird der öffentliche Schlüssel verwendet, bei der Entschlüsselung der private Schlüssel.
<code>KeyFactory.getInstance("RSA")</code>	Diese Methode erstellt eine Instanz der <code>KeyFactory</code> -Klasse für den RSA-Algorithmus. Sie wird verwendet, um ein Schlüsselpaar aus einem <code>byte[]</code> -Array zu rekonstruieren, das zuvor in einer Datei oder einem anderen Format gespeichert wurde.
<code>RSAPublicKey</code> und <code>RSAPrivateKey</code>	Diese Klassen repräsentieren den öffentlichen und den privaten Schlüssel in RSA. Sie ermöglichen den Zugriff auf die einzelnen Komponenten des Schlüssels (z.B. den Modulus n und den Exponenten e oder d).
<code>Signature.getInstance("SHA256withRSA")</code>	Diese Methode wird verwendet, um eine digitale Signatur zu erstellen und zu verifizieren. Das Verfahren verwendet eine Kombination aus einem Hash-Algorithmus (z.B. SHA-256) und dem RSA-Algorithmus, um eine Nachricht zu signieren oder die Signatur zu überprüfen.